Project 9

A Database That Keeps Its Own Books | CIS 4720 Database Management and SQL

Points	Time	Work in	Turn in
10	About 2 hours	club_db_starter.sql	Your GitHub repository URL

Homework. Work on your own. Reference: https://www.cruzjohal.com/database/chapter-5-lab-guide

The due date is listed on Canvas LMS.

What you are building

A small club keeps a list of members and the payments they make. Somebody has to add up what each member has paid, and somebody has to keep a record of every payment that was added or removed. In most systems that somebody is a person, and people forget.

In this project the database does both jobs itself. By the time you finish, inserting one payment row updates a running total and writes a log entry, with no second statement from you.

You build	What it is for	Step
payment	One row per payment, with a surrogate key starting at 5000	2
payment_log	A record of every payment added or removed, written only by triggers	3
member_total	A function that adds up one member's payments	5
Three triggers	Fill in blanks, keep the total current, and write the log	6 to 8

> **This follows In Class Lab 5e**
>
> Everything here is something you did in class, with different table names. The surrogate key starting value is Lab 5e Step 4. The BEFORE INSERT trigger is Lab 5e Step 7. The AFTER DELETE trigger is Lab 5e Step 8. The function is Lab 5d Step 8.
>
> The one genuinely new idea is a trigger doing **two** jobs at once, in Steps 7 and 8. In both of those the second job is written for you, so you can see the shape before you write the first.

> **Label and comment every step**
>
> Each step has a numbered heading already written into club_db_starter.sql. Put your statements under the right one, and put a comment in your own words above every statement saying what it does. It is worth a full point.

Part 1. Set up and build the two tables

Step 1. Connect, create the schema, and run the starter file

Same routine as Lab 5d and Lab 5e, all inside VS Code. You do not need the command prompt at any point in this project.

Make a folder on OneDrive called **CIS4720_Project9**, download **club_db_starter.sql** into it, and open the folder in VS Code. Add a SQLTools connection with the database set to **information_schema**, because it is on every server and so cannot fail.

Open the starter file and run its first two statements, one at a time:

```
show databases;

create database if not exists club_db;
```

Then run the **use club_db;** line that follows them, and add a second SQLTools connection pointing at **club_db** to work from for the rest of the project. Run **SELECT DATABASE();** any time you are unsure which schema your statements are landing in.

Work down the rest of the file. You should end up with four members numbered **100** to **103**, each with total_paid at 0.00.

Before you write anything, read the CREATE TABLE statement for member. It is the model for the next two steps, and the comment above it explains the part you have not written before.

Step 2. Create the payment table, key starting at 5000

Create a table named **payment** with these five columns:

Column	Type	Rules
payment_id	INT	AUTO_INCREMENT, and the primary key
member_id	INT	Never empty. A foreign key to member_id in member
amount	DECIMAL(7,2)	Never empty. Money, so two decimal places
paid_on	DATE	May be left empty. A trigger fills it in later
method	VARCHAR(10)	May be left empty

Make the first generated payment_id be **5000**. The member table shows you how, and Lab 5e Step 4 is the same idea.

Starting the two tables at different numbers is not decoration. A member id is always in the hundreds and a payment id is always in the thousands, so if one ever turns up in the wrong column it is obvious at a glance.

Step 3. Create the payment_log table

This one is written only by triggers. You never insert into it by hand, and it has no foreign key, because a log has to survive the thing it is logging being deleted.

Column	Type	Rules
log_id	INT	AUTO_INCREMENT, and the primary key. Leave this one starting at 1
member_id	INT	May be left empty. No foreign key on this one
amount	DECIMAL(7,2)	May be left empty
action	VARCHAR(20)	May be left empty. Holds ADDED or REMOVED
logged_on	DATETIME	May be left empty. A date and a time, not just a date

> **Why no foreign key here**
>
> Every other link you have built this term used a foreign key, so leaving it off looks like a mistake. It is not.
>
> A foreign key would refuse to let you keep a log entry for a payment that has been deleted, and a log whose entries vanish when the thing they describe vanishes is no log at all. This is the one case where you deliberately do not link the tables.

Step 4. Check your two counters before anything is inserted

Run the counter query from Lab 5e Step 3 against club_db. All three tables should appear, and the numbers tell you the starting values took:

table_name	auto_increment	meaning
member	104	four members already in
payment	5000	nothing inserted yet
payment_log	1	nothing logged yet

If payment says 1 rather than 5000, the starting value did not take. Fix it with ALTER TABLE now, before you insert anything, because the counter only ever moves forward.

Part 2. The function and the three triggers

Each of these four is given to you with its skeleton complete and **one statement removed**. You write the missing statement, and a box under each listing walks you through it a line at a time. Each one is a single statement from CREATE down to END, so select the whole block and run the selection.

> **No DELIMITER in any of these**
>
> Same as Lab 5d and 5e. If you look one of these up and find it wrapped in **DELIMITER $$** and **DELIMITER ;** with **$$** after END, delete all of that. It is an instruction to the command prompt program, not SQL, and the server rejects it with **error 1064**.

Step 5. Create the member_total function

It takes a member id and gives back everything that member has paid.

```
-- adds up one member's payments and hands back the total
CREATE FUNCTION member_total(p_member_id INT)
-- money, so two decimal places
RETURNS DECIMAL(7,2)
DETERMINISTIC
READS SQL DATA
BEGIN
    -- somewhere to hold the answer while we work it out
    DECLARE v_total DECIMAL(7,2);

    -- add up just this member's rows
    -- >>>>>  YOUR STATEMENT GOES HERE  <<<<<

    -- a member with no payments sums to NULL, so turn that into 0.00
    RETURN IFNULL(v_total, 0.00);
END
```

Writing the statement that goes in the gap

One SELECT, and it is unusual in one way: instead of showing you an answer it has to put the answer into the variable. Build it a line at a time.

1. Start with SELECT and the thing you want: the **SUM** of the amount column. One value comes back, not a list, because SUM collapses many rows into one.

2. On the next line write **INTO v_total**. That is the part you have not written before. It says do not display this, store it in the variable that was DECLAREd above.

3. Then FROM, naming the table the amounts live in.

4. Then a WHERE clause so you only add up **one** member's rows. Compare the member_id column against **p_member_id**, the argument the function was handed. Leave the WHERE off and every member gets the club's entire income as their total.

5. Finish it with a semicolon, like every statement inside a BEGIN block.

Step 6. A BEFORE INSERT trigger that fills in what was left out

Exactly the pattern from Lab 5e Step 7, on a different table.

```
-- runs while the row is still on its way in, so it can change it
CREATE TRIGGER trg_payment_before_insert
BEFORE INSERT ON payment
FOR EACH ROW
BEGIN
    -- no date given, so record it as paid today
    -- >>>>>  YOUR STATEMENT GOES HERE  <<<<<

    -- store the method in capitals whatever case it arrived in
    SET NEW.method = UPPER(NEW.method);
END
```

Writing the statement that goes in the gap

This one is a decision, so it needs an IF. Three lines plus its ending.

1. Open with **IF**, then the condition, then **THEN** on the same line. The condition is asking whether paid_on arrived empty, so test **NEW.paid_on IS NULL**. Remember it is IS NULL and never = NULL, which is the Chapter 4 rule that still applies here.

2. On the next line, indented, set the column to today's date. Use **SET NEW.paid_on =** and then the function that gives today's date. You used it in Lab 5e. End that line with a semicolon.

3. Close the decision with **END IF;** on its own line. It takes a semicolon of its own, which is easy to forget.

Why NEW and not the column on its own: NEW is the row on its way in, and putting NEW in front is what tells the server you mean that row rather than something in the table already.

Step 7. An AFTER INSERT trigger that updates and logs

This is the new idea. A trigger body can hold more than one statement, so this one does two jobs every time a payment arrives.

```
-- fires once a payment row has landed
CREATE TRIGGER trg_payment_after_insert
AFTER INSERT ON payment
FOR EACH ROW
BEGIN
    -- job one: bring that member's running total up to date
    -- >>>>>  YOUR STATEMENT GOES HERE  <<<<<

    -- job two: write a line in the log saying what happened
    INSERT INTO payment_log (member_id, amount, action, logged_on)
    VALUES (NEW.member_id, NEW.amount, 'ADDED', NOW());
END
```

Writing the statement that goes in the gap

An UPDATE in three lines. Job two, the INSERT into the log, is already written below it so you can see the shape of a finished trigger body.

1. UPDATE and the name of the table that holds the running total. It is not the table this trigger sits on. A trigger on payment is allowed to change a different table, and that is the whole point of this one.

2. SET total_paid = and then a call to the function you wrote in Step 5, handing it **NEW.member_id** so it adds up the right member. You are not doing any arithmetic yourself. The function recounts the table and returns the answer.

3. A **WHERE** clause matching member_id against NEW.member_id, so only that one member's row changes. Leave it off and every member in the club gets this member's total.

Why NEW works here even though the row is already written: AFTER means the payment has landed, so the function can see it when it adds up. NEW still refers to the row that caused the trigger to fire.

NOW() gives the date and the time together, which is why payment_log uses DATETIME rather than DATE. CURDATE() in Step 6 gives only the date, which is all a payment needs.

Step 8. An AFTER DELETE trigger that does the same in reverse

Nearly identical to Step 7, with two differences worth spotting before you copy it.

```
-- fires once a payment row has gone
CREATE TRIGGER trg_payment_after_delete
AFTER DELETE ON payment
FOR EACH ROW
BEGIN
    -- OLD, not NEW, because the row that matters is the one
    -- that was just removed. a deleted row has no NEW
    -- >>>>>  YOUR STATEMENT GOES HERE  <<<<<

    -- and the log says REMOVED this time
    INSERT INTO payment_log (member_id, amount, action, logged_on)
    VALUES (OLD.member_id, OLD.amount, 'REMOVED', NOW());
END
```

Writing the statement that goes in the gap

Write the same three lines you wrote in Step 7, with one word changed throughout.

1. UPDATE the same table, SET the same column, using the same function, with the same WHERE clause.

2. Everywhere Step 7 said **NEW**, write **OLD** instead. That is both places: the argument you hand the function, and the value in the WHERE clause.

3. Nothing else changes. The function recounts whatever is left in payment right now, so the same call gives the right answer after a row is added and after one is taken away.

Get this wrong and the server tells you plainly: **error 1363, there is no NEW row in on delete trigger**. A deleted row has no NEW, only OLD.

> **The two differences, and why they matter**
>
> **OLD instead of NEW.** There is no NEW row in a delete trigger. Use NEW here and the server refuses with **error 1363**.
>
> **The log says REMOVED.** Without that word the log would record a deletion as if it were a payment, which is worse than not logging it at all.
>
> Notice what is NOT different: the UPDATE is the same. The function recounts whatever is in payment right now, so it gives the right answer after a row is added and after one is taken away, with no arithmetic of its own.

Part 3. Prove the whole thing works

Step 9. Add four payments, then remove one

You write these. Insert the four rows below, naming only the columns shown, so the Step 6 trigger has something to fill in on two of them.

member_id	amount	paid_on	method
100	45.00	leave it out	cash
100	30.00	2026-04-02	card
101	45.00	leave it out	check
102	60.00	2026-03-15	card

Then select the payment table. Four things should have happened without you asking for any of them:

payment_id	member_id	amount	paid_on	method
5000	100	45.00	today's date	CASH
5001	100	30.00	2026-04-02	CARD
5002	101	45.00	today's date	CHECK
5003	102	60.00	2026-03-15	CARD

Now select the member table. The totals maintained themselves:

member_id	first_name	total_paid
100	Noor	75.00
101	Caleb	45.00
102	Sofia	60.00
103	Desmond	0.00

Noor shows 75.00 because she made two payments and the function added them up. Desmond is still on 0.00 because he has made none, and the IFNULL in the function is what made that 0.00 rather than empty.

Now delete the 30.00 payment, the one with payment_id 5001. Then look at Noor again and look at the log.

What to check	What you should see
Noor's total_paid	Back down to 45.00, corrected by the Step 8 trigger
payment_log	Five rows: four ADDED from Step 7, then one REMOVED from Step 8

What to check	What you should see
log_id values	1 through 5, because payment_log started its counter at 1

Read that log. You never wrote a single INSERT into payment_log, and yet there is a complete record of every payment that came and went, with the time it happened. That is what this project is about.

Part 4. Push to GitHub

Step 10. Publish to a new repository

Save club_db_starter.sql first. If you are not signed in to GitHub, click **Accounts** at the bottom of the left bar, choose **Sign in with GitHub**, and click **Authorize**.

1. Click **Source Control**, then **Publish to GitHub**.
2. Name it **cis4720-project9**. It has to be a name you have not used.
3. Choose **Publish to a public repository**. A private one gives your instructor a 404 page instead of your work.
4. Tick **club_db_starter.sql**. Leave the **.vscode** folder unticked, because that is the one holding your password.

Step 11. Check it and submit the URL

1. Open the repository on github.com, click the file and read it on the page. Check all eight of your steps are there with your comments above them. What is on this page is exactly what gets graded.
2. Open the URL in a private or incognito window. A **404** means it is private, so go to **Settings** and change the visibility to public.
3. Paste the URL into the Project 9 submission box on Canvas.

This is homework, so work on your own. Looking syntax up in your textbook, the lab guide or your Lab 5e file is expected. Copying another student's file is not, and two identical submissions are easy to spot.

How it is graded

Steps	What is being checked	Points
1	Schema created and the starter file running, four members at 100 to 103	0.5
2	payment created with its key starting at 5000 and a foreign key to member	2.0
3	payment_log created, with no foreign key on it	1.5
4	The counter query run, showing 104, 5000 and 1	1.0
5	The member_total function created	1.0
6	The BEFORE INSERT trigger created	1.0
7 to 8	Both AFTER triggers created, each doing its two jobs	1.5
9	Four payments in, one removed, and the log showing all five lines	0.5
All	Required. A comment in your own words above every statement	1.0
10 to 11	A new public repository with the file in it, URL submitted	0.0

Total 10 points. The due date is listed on Canvas LMS.

If something goes wrong

What you see	What it means
ERROR 1064 near '--something' in a trigger or function	The space after the two dashes went missing in the paste. MySQL only treats -- as a comment when a space or a tab follows it, so --no is read as two minus signs. Put the space back, or use # instead, which needs no space.
ERROR 1064 near 'DELIMITER $$'	You copied something that still has the DELIMITER lines around it. Delete them and the $$ after END.
ERROR 1064 on the DECLARE, SET or IF line	Your editor split the block at a semicolon inside it. Select the whole block from CREATE to END and run the selection.
ERROR 1363, there is no NEW row in on delete trigger	Step 8 needs OLD, not NEW. A deleted row has no NEW.
ERROR 1362, updating of NEW row is not allowed	SET NEW.something only works in a BEFORE trigger. Steps 7 and 8 are AFTER triggers and do not set anything on the row.
ERROR 1452, cannot add or update a child row	A payment names a member_id that is not in member. The members are 100 to 103, not 1 to 4.
ERROR 1305, FUNCTION club_db.member_total does not exist	Step 5 did not run, or it ran in a different schema. Check SELECT DATABASE();
ERROR 1359, trigger already exists	You ran a trigger step twice. Run DROP TRIGGER and the name, then create it again.
payment ids start at 1, not 5000	The starting value did not take. Fix it with ALTER TABLE, then delete the rows and insert them again.
total_paid stays at 0.00	The Step 7 trigger did not run, or it was created after you inserted the payments. A trigger only affects rows written after it exists.

Stuck on syntax? The course lab guide works through all of it: https://www.cruzjohal.com/database/chapter-5-lab-guide